

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

ANS :

Scenario 1

```
SET SERVEROUTPUT ON;
```

```
CREATE TABLE Customers (  
    CustomerID NUMBER PRIMARY KEY,  
    Name VARCHAR2(50),  
    Age NUMBER,  
    Balance NUMBER,  
    IsVIP VARCHAR2(5)  
);
```

```
CREATE TABLE Loans (  
    LoanID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    InterestRate NUMBER,  
    DueDate DATE  
);
```

```
INSERT INTO Customers VALUES (101,'Rahul',65,15000,'FALSE');  
INSERT INTO Customers VALUES (102,'Priya',45,8000,'FALSE');  
INSERT INTO Customers VALUES (103,'Amit',70,20000,'FALSE');
```

```
INSERT INTO Loans VALUES (1,101,8.5,SYSDATE+20);
INSERT INTO Loans VALUES (2,102,9.0,SYSDATE+50);
INSERT INTO Loans VALUES (3,103,7.5,SYSDATE+15);
```

```
COMMIT;
```

```
BEGIN
```

```
  FOR c IN (SELECT CustomerID, Age FROM Customers) LOOP
```

```
    IF c.Age > 60 THEN
```

```
      UPDATE Loans
```

```
        SET InterestRate = InterestRate - 1
```

```
        WHERE CustomerID = c.CustomerID;
```

```
    END IF;
```

```
  END LOOP;
```

```
COMMIT;
```

```
  DBMS_OUTPUT.PUT_LINE('Scenario 1: Loan interest updated successfully.');
```

```
END;
```

```
/
```

```
SELECT * FROM Loans;
```

Scenario 2

```
SET SERVEROUTPUT ON;
```

```
CREATE TABLE Customers (  
  CustomerID NUMBER PRIMARY KEY,  
  Name VARCHAR2(50),  
  Age NUMBER,  
  Balance NUMBER,  
  IsVIP VARCHAR2(5)  
);
```

```
INSERT INTO Customers VALUES (101,'Rahul',65,15000,'FALSE');  
INSERT INTO Customers VALUES (102,'Priya',45,8000,'FALSE');  
INSERT INTO Customers VALUES (103,'Amit',70,20000,'FALSE');
```

```
COMMIT;

BEGIN

    FOR c IN (SELECT CustomerID, Balance FROM Customers) LOOP

        IF c.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP='TRUE'
            WHERE CustomerID=c.CustomerID;
        END IF;

    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Scenario 2: VIP customers updated.');
```

END;

/

```
SELECT * FROM Customers;
```

Scenario 3

```
SET SERVEROUTPUT ON;

CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(50),
    Age NUMBER,
    Balance NUMBER,
    IsVIP VARCHAR2(5)
);

CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    InterestRate NUMBER,
    DueDate DATE
```

```
);
```

```
INSERT INTO Customers VALUES (101,'Rahul',65,15000,'FALSE');
```

```
INSERT INTO Customers VALUES (102,'Priya',45,8000,'FALSE');
```

```
INSERT INTO Customers VALUES (103,'Amit',70,20000,'FALSE');
```

```
INSERT INTO Loans VALUES (1,101,8.5,SYSDATE+20);
```

```
INSERT INTO Loans VALUES (2,102,9.0,SYSDATE+50);
```

```
INSERT INTO Loans VALUES (3,103,7.5,SYSDATE+10);
```

```
COMMIT;
```

```
BEGIN
```

```
FOR l IN (
```

```
  SELECT c.Name, lo.DueDate
```

```
  FROM Customers c
```

```
  JOIN Loans lo
```

```
  ON c.CustomerID = lo.CustomerID
```

```
  WHERE lo.DueDate BETWEEN SYSDATE AND SYSDATE+30
```

```
) LOOP
```

```
  DBMS_OUTPUT.PUT_LINE('Reminder: Dear '
```

```
  || l.Name
```

```
  || ', your loan is due on '
```

```
  || TO_CHAR(l.DueDate,'DD-MON-YYYY'));
```

```
END LOOP;
```

```
END;
```

```
/
```

Output:

Scenario 1: Loan interest updated successfully.

LOANID CUSTOMERID INTERESTRATE DUEDATE				

1	101	7.5	16-JUL-26	
2	102	9	15-AUG-26	
3	103	6.5	11-JUL-26	
LOANID CUSTOMERID INTERESTRATE DUEDATE				

1	101	7.5	16-JUL-26	
2	102	9	15-AUG-26	
3	103	6.5	11-JUL-26	

Output:

Scenario 2: VIP customers updated.

CUSTOMERID NAME		AGE

BALANCE	ISVIP	

101	Rahul	65
15000	TRUE	
102	Priya	45
8000	FALSE	
103	Amit	70
20000	TRUE	
CUSTOMERID NAME		AGE

BALANCE	ISVIP	

101	Rahul	65
15000	TRUE	
102	Priya	45
8000	FALSE	

Output:

Reminder: Dear Rahul, your loan is due on 16-JUL-2026

Reminder: Dear Amit, your loan is due on 06-JUL-2026

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

ANS :

Scenario 1: Process Monthly Interest

SET SERVEROUTPUT ON;

```
CREATE TABLE Accounts (  
    AccountID NUMBER PRIMARY KEY,  
    CustomerName VARCHAR2(50),  
    AccountType VARCHAR2(20),  
    Balance NUMBER
```

```
);
```

```
INSERT INTO Accounts VALUES (101,'Rahul','Savings',10000);
```

```
INSERT INTO Accounts VALUES (102,'Priya','Current',15000);
```

```
INSERT INTO Accounts VALUES (103,'Amit','Savings',20000);
```

```
COMMIT;
```

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
```

```
AS
```

```
BEGIN
```

```
    UPDATE Accounts
```

```
    SET Balance = Balance + (Balance * 0.01)
```

```
    WHERE AccountType = 'Savings';
```

```
    COMMIT;
```

```
END;
```

```
/
```

```
BEGIN
```

```
    ProcessMonthlyInterest;
```

```
    DBMS_OUTPUT.PUT_LINE('Scenario 1: Monthly interest processed.');
```

```
END;
```

```
/
```

```
BEGIN
```

```
    FOR a IN (SELECT * FROM Accounts) LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(
```

```
            a.AccountID || ' ' ||
```

```
            a.CustomerName || ' ' ||
```

```
            a.AccountType || ' ' ||
```

```
            a.Balance
```

```
        );
```

```
    END LOOP;
```

```
END;
```

```
/
```

Scenario 2: Update Employee Bonus

```
SET SERVEROUTPUT ON;
```

```
CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    EmployeeName VARCHAR2(50),  
    Department VARCHAR2(30),  
    Salary NUMBER  
);
```

```
INSERT INTO Employees VALUES (1,'Arun','IT',50000);  
INSERT INTO Employees VALUES (2,'Kiran','HR',40000);  
INSERT INTO Employees VALUES (3,'Sneha','IT',60000);
```

```
COMMIT;
```

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
    p_department VARCHAR2,  
    p_bonus NUMBER  
)  
AS  
BEGIN  
    UPDATE Employees  
    SET Salary = Salary + (Salary * p_bonus / 100)  
    WHERE Department = p_department;  
  
    COMMIT;  
END;  
/
```

```
BEGIN  
    UpdateEmployeeBonus('IT',10);  
    DBMS_OUTPUT.PUT_LINE('Scenario 2: Employee bonus updated.');
```

```
END;
```

```
/
```

```
BEGIN  
    FOR e IN (SELECT * FROM Employees) LOOP  
        DBMS_OUTPUT.PUT_LINE(  
            e.EmployeeID || ' ' ||
```



```

        e.EmployeeName || ' ' ||
        e.Department || ' ' ||
        e.Salary
    );
END LOOP;
END;
/

```

Scenario 3: Transfer Funds

```
SET SERVEROUTPUT ON;
```

```

CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerName VARCHAR2(50),
    Balance NUMBER
);

```

```

INSERT INTO Accounts VALUES (101,'Rahul',10000);
INSERT INTO Accounts VALUES (102,'Priya',8000);

```

```
COMMIT;
```

```

CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from NUMBER,
    p_to NUMBER,
    p_amount NUMBER
)

```

```
AS
```

```

    v_balance NUMBER;
BEGIN

```

```

    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from;

```

```
IF v_balance >= p_amount THEN
```

```

    UPDATE Accounts

```

```
SET Balance = Balance - p_amount  
WHERE AccountID = p_from;
```

```
UPDATE Accounts  
SET Balance = Balance + p_amount  
WHERE AccountID = p_to;
```

```
COMMIT;
```

```
DBMS_OUTPUT.PUT_LINE('Transfer Successful');
```

```
ELSE
```

```
DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
```

```
END IF;
```

```
END;
```

```
/
```

```
BEGIN
```

```
TransferFunds(101,102,3000);
```

```
END;
```

```
/
```

```
BEGIN
```

```
FOR a IN (SELECT * FROM Accounts) LOOP
```

```
DBMS_OUTPUT.PUT_LINE(
```

```
a.AccountID || ' ' ||
```

```
a.CustomerName || ' ' ||
```

```
a.Balance
```

```
);
```

```
END LOOP;
```

```
END;
```

```
/
```

Output:

Scenario 1: Monthly interest processed.

101 Rahul Savings 10100

102 Priya Current 15000

103 Amit Savings 20200

Output:

Scenario 2: Employee bonus updated.

1 Arun IT 55000

2 Kiran HR 40000

3 Sneha IT 66000

Output:

Transfer Successful

101 Rahul 7000

102 Priya 11000